EXPERIMENT - 50

1. Problem Statement

Write a C program that reads the size and elements of a variable-length integer array, nums[]. Pass the array to a function named majority_element() using its address. Use pointers to pass the array between the calling function and majority_element(). The function majority_element() must:

- Count the occurrences of each distinct element in nums[].
- Find the element that appears strictly more than floor(size / 2) times.
- Display the majority element if it exists.
- Display −1 if no majority element exists.

Note: the majority element is not necessarily the element with the highest frequency. It must occur strictly more than floor(size / 2) times.

For 3 3 2 3 3 the answer is 3, since floor(5 / 2) is 2 and 3 occurs four times. For 2 2 1 1 1 2 2 the answer is 2, since floor(7 / 2) is 3 and 2 occurs four times. For 1 2 3 2 3 3 the answer is −1, since floor(6 / 2) is 3 and 3 occurs only three times.

2. Algorithm

Main Program

Step 1: START

Step 2: Read n and declare the variable length array nums of size n.

Step 3: Read the n elements of nums.

Step 4: Call majority_element(&nums[0], n).

Step 5: STOP

Function majority_element()

Step 1: Set half = n / 2.

Step 2: Repeat for i = 0 to n − 1:

Set count = 0.

Repeat for j = 0 to n − 1: if nums[j] is equal to nums[i], add 1 to count.

If count is greater than half, display nums[i] and return to the calling function.

Step 3: Display −1, since no majority element was found.

Step 4: Return to the calling function.

3. Test Case Table

#	Array	Expected Output	Actual Output	Result
1	3 3 2 3 3	Majority element: 3, as half is 2 and 3 occurs four times	Matches example 1 in the question	**PASS**

#	Array	Expected Output	Actual Output	Result
2	2 2 1 1 1 2 2	Majority element: 2, as half is 3 and 2 occurs four times	Matches example 2	**PASS**
3	1 2 3 2 3 3	−1, as 3 occurs three times and half is also 3	−1, so three is not more than three	**PASS**
4	1 1 1 2 2 3	−1, even though 1 has the highest frequency	−1, which is the point of the note	**PASS**
5	7	Majority element: 7, as half is 0 and 7 occurs once	7	**PASS**
6	4 1 4 2 4 3 4	Majority element: 4, spread out instead of together	4, so the copies need not be next to each other	**PASS**
7	1 1 2 2	−1, both values tie at two	−1	**PASS**

4. Source Code

majority50.c

```
/* Experiment 42. One function for each of the three operations.
   Every option reads its own input here, which is different from what
   I have been doing. Option 3 needs two strings and the other two need
   one, so there was no single string I could read at the start that
   would have suited all three.
   The question lists only three operations, but a do while menu has no
   way of stopping, so option 4 is mine. Without it the menu keeps
   coming back and the only way out is to kill the program. */

#include <stdio.h>

void removeVowels(const char s[], char result[]);
char firstRepeatingLowercase(const char s[]);
int areAnagrams(const char s1[], const char s2[]);

int main(void)
{
    char str[100], str2[100], result[100];
    char ch;
    int choice;

    do {
        printf("\nMENU\n");
        printf("1. Remove all the vowels from a string\n");
        printf("2. Find the first repeating lowercase letter\n");
        printf("3. Check whether two strings are anagrams\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter a string: ");
                scanf(" %[^\n]", str);
                removeVowels(str, result);
                printf("String without vowels: %s\n", result);
                break;

            case 2:
                printf("Enter a string: ");
                scanf(" %[^\n]", str);
                ch = firstRepeatingLowercase(str);
                if (ch == '\0')
                    printf("No lowercase letter repeats in this string\n");
                else
                    printf("First repeating lowercase letter: %c\n", ch);
                break;

            case 3:
                printf("Enter the first string: ");
                scanf(" %[^\n]", str);
                printf("Enter the second string: ");
                scanf(" %[^\n]", str2);
                if (areAnagrams(str, str2))
                    printf("The two strings are anagrams\n");
                else
                    printf("The two strings are not anagrams\n");
                break;

            case 4:
                printf("Exiting the program.\n");
                break;
        }
    } while (choice != 4);

    return 0;
}
```

```
/* i walks through the old string and j marks where to write in the new
   one, and they drift apart because j only moves when something is
   actually copied.
   The '\0' on the last line is the one I forgot. Programming in C came
   out right and then had junk stuck on the end of it, because printf
   had no idea where the new string finished.
   Both cases are listed because A and a are different characters, and
   leaving the capitals out would have let the A through in Day. */
void removeVowels(const char s[], char result[])
{
    int i, j = 0;
    char ch;

    for (i = 0; s[i] != '\0'; i++) {
        ch = s[i];

        if (ch != 'a' && ch != 'e' && ch != 'i' && ch != 'o' && ch != 'u' &&
            ch != 'A' && ch != 'E' && ch != 'I' && ch != 'O' && ch != 'U') {
            result[j] = ch;
            j++;
        }
    }

    result[j] = '\0';
}

/* First repeating can be read two ways, and the two give different
   answers. In abcdbca one reading says b, the first letter you meet
   that you have seen before. The other says a, because out of the
   letters that repeat at all, a is the one that showed up earliest.
   I went with the second, which is the usual meaning, and that is why
   the counting happens first and the walk from the left happens after.
   One single pass would have given the other answer.
   count has 26 boxes, one per small letter, and s[i] - 'a' turns a
   letter straight into its box number, since 'a' is 97 and 'b' is 98.
   Capitals are skipped in both loops, so AABBcdc answers c, even though
   A and B repeat more times than c does. */
char firstRepeatingLowercase(const char s[])
{
    int count[26] = {0};
    int i;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z')
            count[s[i] - 'a']++;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z' && count[s[i] - 'a'] > 1)
            return s[i];

    /* '\0' is not a letter, so main can use it to mean nothing was
       found. */
    return '\0';
}

/* Anagrams hold the same characters the same number of times, only in
   a different order. My first idea was to sort both strings and see if
   they came out identical, but counting turned out to be less code, and
   it leaves the two strings alone instead of rearranging them.
   256 boxes covers every character there is, so spaces and digits are
   counted along with the letters, and nothing is quietly ignored.
   The length test is not only a shortcut. The counts could agree on
   every character and the strings still be different lengths if one of
   them held something the other did not. */
int areAnagrams(const char s1[], const char s2[])
{
    int count1[256] = {0};
    int count2[256] = {0};
    int i, len1 = 0, len2 = 0;

    for (i = 0; s1[i] != '\0'; i++) {
        count1[(unsigned char) s1[i]]++;
        len1++;
```

```
    }

    for (i = 0; s2[i] != '\0'; i++) {
        count2[(unsigned char) s2[i]]++;
        len2++;
    }

    if (len1 != len2)
        return 0;

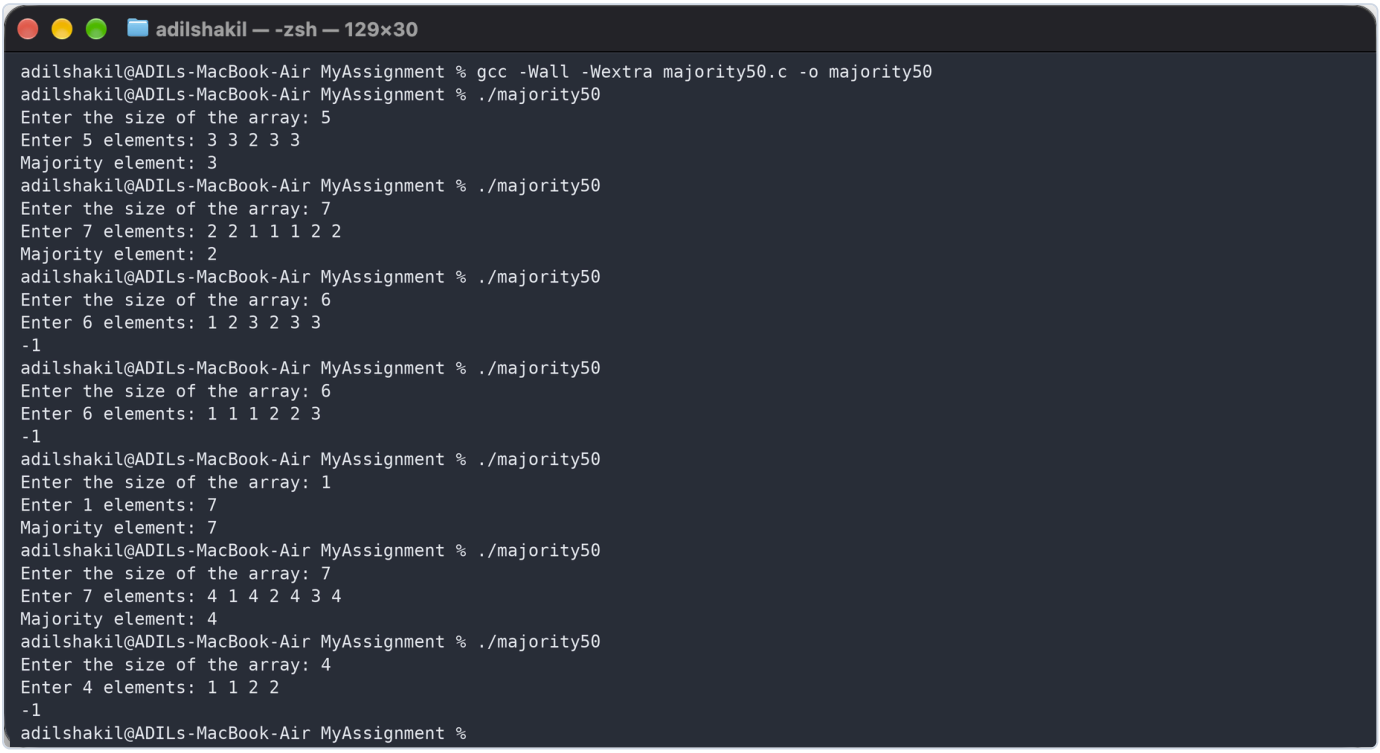
    for (i = 0; i < 256; i++)
        if (count1[i] != count2[i])
            return 0;

    return 1;
}
```

5. Compilation and Output

Compiled with `gcc -Wall -Wextra majority50.c -o majority50` so the small warnings would show up too. It built clean.

All Seven Runs



All seven cases came out right, and the first three are the examples given in the question.

majority_element() takes `int *nums`, so it is handed the address of the array and not a copy of it, and the call is written as `&nums[0]`.

The floor in the question sorts itself out. `n / 2` on two integers is integer division, which already throws away the decimal part, so 7 / 2 gives 3 and not 3.5. There is nothing extra to do.

The word strictly is the part that actually matters. The test is `count > half` and not `count >= half`. Case 3 is where it shows: 3 appears three times and half is also 3, so three is not more than three and the answer is −1.

Case 4 is the one I added for the note at the bottom of the question. In 1 1 1 2 2 3 the value 1 has the highest frequency of the three, but half is 3 and 1 only reaches 3, so it still is not a majority. Having the most copies is not the same as having more than half of them.

Case 6 checks something the three examples do not. The copies of 4 are scattered through the array rather than sitting together, and the counting loop does not care, because it runs over the whole array for each element.